SMM: The Mechanics and the Mess

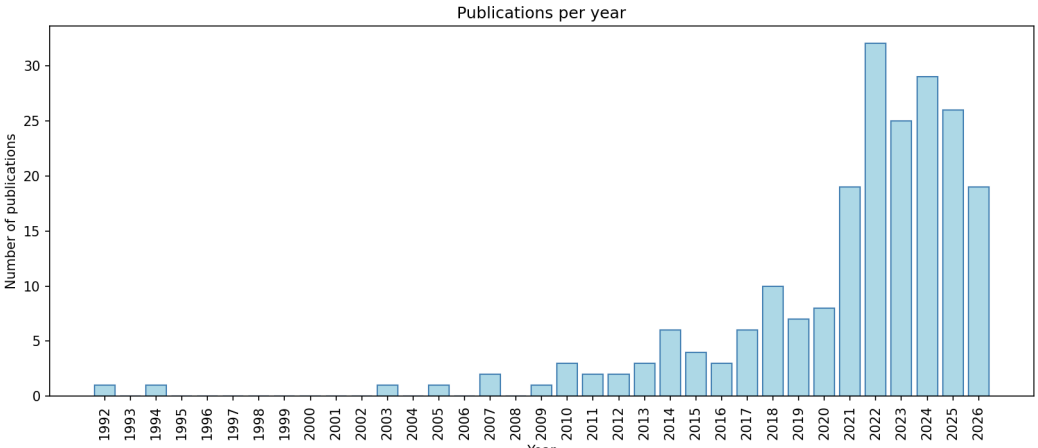
DSE Summer School

Toni M. Whited

August 2026

Copyright ©Toni M. Whited

Structural Finance is Not Dead. Not Even Close.



Copyright ©Toni M. Whited

The premise of this lecture

- I am going to assume that you can solve and simulate a dynamic model.
- Or that you can get an AI to do it for you.
- Ten years ago, half of a course like this was model solution techniques. That part of the job is now cheap.
- **Estimation is not cheap.**
- It is a messy engineering project, it requires human input, and the failures are silent.
- Today: the mechanics of SMM (which are simple) and the mess (which is where papers die).

Copyright ©Toni M. Whited

Division of labor between you and the machine

- AI is genuinely good at:
  - Writing a model solver and a simulator
  - Translating your slow Matlab into fast Julia, C, or Fortran
  - Making any code you have faster
  - Boilerplate: numerical derivatives, parallelization, plots
- AI is genuinely bad at:
  - Noticing that the code *runs* but is *wrong*
  - Choosing moments that identify your parameters
  - Telling you that your model is misspecified in an interesting way
  - Caring whether your estimate of a fixed cost is economically absurd
- **You are the principal investigator. The AI is a tireless, overconfident, and fallible RA.**
- Everything in this lecture is organized around that division of labor.

Copyright ©Toni M. Whited

The plan for the next three hours

- Hour 1: The objective function
  - What SMM is, why it works, and its asymptotic distribution
- Hour 2: The weight matrix and identification
  - Influence functions: one tool for every covariance matrix you will ever need
  - How to tell whether your model is identified
- Hour 3: The engineering project
  - The pipeline, the optimizer, the seeds, the verification
  - All of my accumulated “tips,” which are no longer tips. They are the job.

Outline

Copyright ©Toni M. Whited

A very simple investment model

- Partial equilibrium investment model cast in discrete time, with an infinite horizon.
- Risk-neutral managers choose the capital stock each period to maximize the value of the firm: the expected present value of the stream of cash flows to (or from) shareholders.
- Cash flows are given by:

$$e\left(k, I, z\right) \equiv \pi(k, z) - I$$

- k is the capital stock, and I is investment, with

$$k' = (1 - \delta)k + I,$$

  where a prime indicates a variable in the subsequent period, and δ is the depreciation rate.
- $\pi(k, z) = zk^{\alpha}$, $\alpha < 1$ is a profit function.
- z is a profit shock that follows a Markov process with transition probability $q(z', z)$.

The Bellman equation

- Bellman's observation: you can solve this problem recursively. The value of the firm today is the current cash flow plus the discounted expected value of the firm tomorrow:

$$V(k,z) = \max_{k'} \left\{ zk^{\alpha} - (k' - (1-\delta)k) + \frac{1}{1+r} \int V(k',z')\, dq(z',z) \right\}$$

- r is the risk-free interest rate.

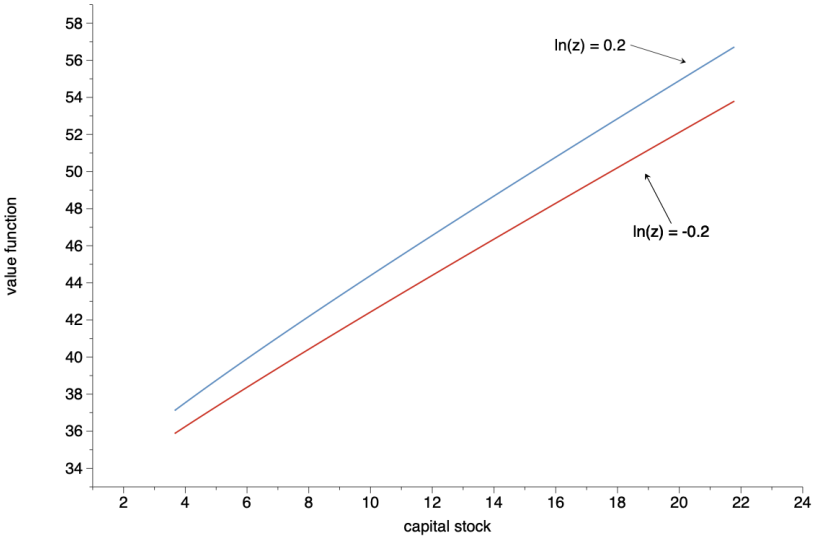
- This is the object that your code (or your AI's code) solves, usually by value function iteration on a grid for (k, z).

- I am not going to teach that today. What I need you to internalize is the **output** of that computation.

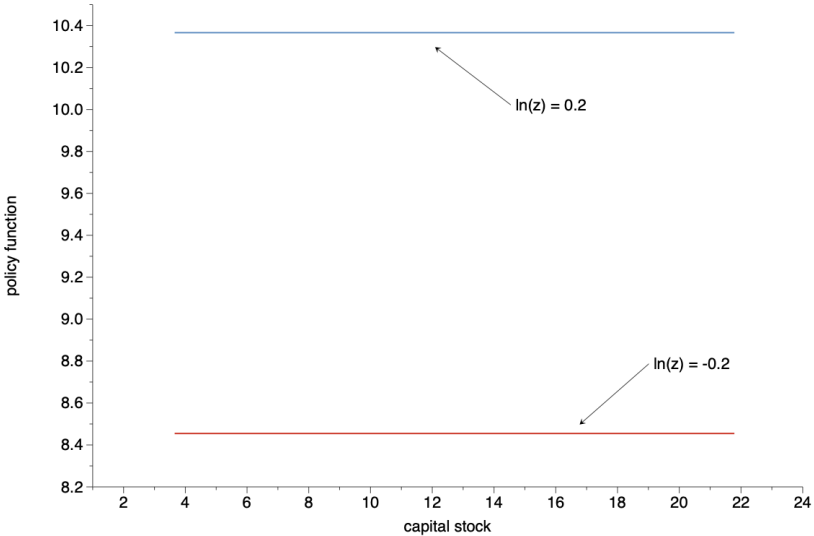
The solution is a pair of functions, not a number

- The state of the firm is the pair (k, z): what it owns, and how profitable it currently is.
- Solving the Bellman equation gives you **two functions defined on the state space**:
  - The value function, $V(k, z)$: what the firm is *worth* in each state.
  - The policy function, $k' = g(k, z)$: what the firm *does* in each state: the optimal capital stock for tomorrow.
- The policy function is the model's complete behavioral prediction. Every simulated data point you will ever match to real data comes from evaluating g.
- In practice both objects are arrays: one number per grid point (k, z).

Value Function



Policy Function



Simulation is just running the policy function

- With the policy function you can simulate the model.
- Start with some arbitrary capital stock and z shock.
- Use the policy function to find the new capital stock.
- Use the transition matrix (and a random draw) to generate a new z shock.
- Repeat.
- Do this for many firms and many periods, and you have a simulated panel that looks just like the panels you work with, except that **you** chose the parameters that generated it.

Outline

Copyright ©Toni M. Whited

When do you need a simulation estimator?

- You have an economic model with parameters θ.
- The model implies restrictions on the data: means, variances, covariances, regression slopes, policy functions.
- If you could write these restrictions down analytically, you would just do GMM or MLE and go home.
- But in most interesting dynamic models you **cannot**. The mapping from θ to the data passes through a value function and a policy function, and neither has a closed form.
- You can, however, **simulate** the model: feed in θ, get out fake data.
- SMM: choose θ so that the fake data look like the real data.

Copyright ©Toni M. Whited

What if . . . you wanted to estimate a mean?

- Suppose you have an $i.i.d.$ sample, y_i, of length n.
- Classical method-of-moments: pick an estimate, μ, based on the moment condition

$$E\left(y - \mu\right) = 0$$

- The sample counterpart is

$$\frac{1}{n}\sum_{i=1}^{n} y_i - \hat{\mu} = 0$$

- Here you have a closed-form expression for the moment condition.
- But you might not . . .

A very convoluted way to estimate a mean

- Generate a random vector with a mean μ. Calculate its average. Call it $\hat{\mu}_1$.
- Do this S times and calculate

$$\tilde{\mu} = \frac{1}{S} \sum_{s=1}^{S} \hat{\mu}_s$$

- Pick the μ that sets

$$\tilde{\mu} - \frac{1}{n} \sum_{i=1}^{n} y_i$$

  as close to zero as possible.

- This **is** a simulated method of moments estimator.
- The example is silly because we know the closed form. But swap "generate a random vector with mean μ" for "solve and simulate a dynamic model with parameters θ," and it stops being silly.

The silly example contains the whole method

- Compute statistics on real data.
- Compute the *exact same* statistics on simulated data.
- Choose parameters to make the two sets of statistics close.
- Two questions make this rigorous:
  - What does “close” mean? The weight matrix.
  - How much noise does the simulation add? The $(1 + 1/S)$ factor.
- That is the entire econometric content of SMM. Everything else is engineering and economics.

Outline

Copyright ©Toni M. Whited

Setup

- Let w_i be an $i.i.d.$ data vector, $i = 1, \ldots, n$.
- Let $y_{is}(\theta)$ be an $i.i.d.$ simulated vector from simulation s, $i = 1, \ldots, n$, and $s = 1, \ldots, S$.
- The simulated data vector, $y_{is}(\theta)$, depends on a vector of structural parameters, θ.
- The goal is to estimate θ by matching a set of *simulated moments*, denoted as $g(y_{is}(\theta))$, with the corresponding set of actual *data moments*, denoted as $\mathbb{E}(g(w_i))$.
- The simulated moments are functions of θ because the moments differ depending on the choice of θ.

Moment matching

- First step: estimate $\mathbb{E}\left(g\left(w_i\right)\right)$ using the actual data.
- Second step: construct S simulated data sets based on a given parameter vector.
- For each simulated data set, compute a simulated moment, $g\left(y_{is}\left(\theta\right)\right)$.
- **You have to make the *exact* same calculations on the simulated data as you do on the real data.**
  - Same variable definitions, same trimming, same fixed effects, same everything.
  - Best practice: **one** moment-calculating function, called on two different datasets.
- $Sn > n$. Michaelides and Ng (2000) find that good finite sample performance requires a simulated sample approximately ten times as large as the actual data sample.

Now let's figure out how to match the moments

- Define

$$g_n(\theta) = n^{-1} \sum_{i=1}^{n} \left[g(w_i) - S^{-1} \sum_{s=1}^{S} g(y_{is}(\theta)) \right].$$

- The simulated moments estimator of θ is then defined as the solution to the minimization of

$$\hat{\theta} = \arg\min_{\theta} Q(\theta, n) \equiv g_n(\theta)' \hat{W}_n g_n(\theta),$$

- $\hat{W}_n$ is a positive definite matrix that converges in probability to a deterministic positive definite matrix W.

- This quadratic form should look extremely familiar. SMM is GMM in which one input to the moment function comes off a simulation.

SMM is a special case of GMM, so . . . let's review GMM

- GMM is based on moment restrictions:

$$E\left(\boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}_0\right)\right) = 0$$

- The estimator minimizes a quadratic form in the sample analogues:

$$\boldsymbol{Q}_n\left(\boldsymbol{\theta}\right) = \left[n^{-1}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right]' \widehat{\boldsymbol{\Xi}} \left[n^{-1}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right]$$

  where $\widehat{\boldsymbol{\Xi}}$ is a positive definite weight matrix that converges in probability to $\boldsymbol{\Xi}_0$.

- The formal theory for the simulated version is in Pakes and Pollard (1989), Lee and Ingram (1991), and Duffie and Singleton (1993).

- The punchline of all three papers: everything you know about GMM goes through, with one small correction for simulation error.

- So we are going to derive (informally) the asymptotic distribution of a GMM estimator, and the SMM results will fall out for free.

Notation: the Jacobian

- Define

$$\begin{aligned} \boldsymbol{G}' &= \frac{\partial \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}'} \\ \boldsymbol{G}_0' &= E\left(\boldsymbol{G}'\right) \end{aligned}$$

- $\boldsymbol{G}'$ is the Jacobian matrix and is a function of the data:

$$\partial \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right) / \partial \boldsymbol{\theta}' \equiv \begin{bmatrix} \partial g_1/\partial \theta_1 & \partial g_1/\partial \theta_2 & \dots & \partial g_1/\partial \theta_k \\ \vdots & \vdots & \ddots & \vdots \\ \partial g_m/\partial \theta_1 & \partial g_m/\partial \theta_2 & \dots & \partial g_m/\partial \theta_k \end{bmatrix}$$

- With k parameters and m moments, $\boldsymbol{G}$ is $k \times m$, and $\boldsymbol{G}'$ is $m \times k$.

- Remember this object. Its rank decides whether you are identified, its numerical computation is a landmine, and it is the backbone of every formula for the rest of the day.

The plan: linearize the first-order condition

- We will derive the asymptotic distribution of a GMM estimator.
- We will do this by linearizing, so that the GMM estimator is not the argmin of a complicated function.
- Instead, we will express it as a closed-form function of sample averages, up to a term that converges in probability to zero.
- That linearization is the influence function, and in Hour 2 it will hand us every weight matrix and standard error we need.

Asymptotic Distribution of GMM Estimators

- The asymptotic distribution of $\sqrt{n}\left(\widehat{\boldsymbol{\theta}} - \boldsymbol{\theta}_0\right)$ is $N\left(0, \boldsymbol{V}\right)$, in which

$$\begin{array}{rcl} \boldsymbol{V} & \equiv & \left[\boldsymbol{G}_0 \boldsymbol{\Lambda}^{-1} \boldsymbol{G}_0'\right]^{-1} \\ & & (k \times m)(m \times m)(m \times k) \end{array}$$

- Derivation: Recall that we are trying to minimize:

$$\boldsymbol{Q}_n\left(\boldsymbol{\theta}\right) = \left[n^{-1} \sum_{i=1}^{n} \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right]' \widehat{\boldsymbol{\Xi}} \left[n^{-1} \sum_{i=1}^{n} \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right]$$

- How do you minimize anything? Take the derivative w.r.t $\boldsymbol{\theta}$ and set the result equal to zero.

Copyright ©Toni M. Whited

Asymptotic Distribution of GMM Estimators

- Take the derivative and set it to zero.

$$\begin{aligned} \partial \boldsymbol{Q}_n\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right) / \partial \boldsymbol{\theta} &= 0 \\ 2\left[n^{-1}\sum_{i=1}^{n} \boldsymbol{G}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right] \widehat{\boldsymbol{\Xi}} \left[n^{-1}\sum_{i=1}^{n} \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right] &= 0 \\ \left[n^{-1}\sum_{i=1}^{n} \boldsymbol{G}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right] \widehat{\boldsymbol{\Xi}} \left[n^{-1}\sum_{i=1}^{n} \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)\right] &= 0 \end{aligned}$$

- Now take a **mean**-value (not Taylor) expansion of $\sum_{i=1}^{n} \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right)$

$$\sum_{i=1}^{n} \boldsymbol{g}\left(\boldsymbol{w}_i, \boldsymbol{\theta}\right) = \sum_{i=1}^{n} \boldsymbol{g}\left(\boldsymbol{w}_i, \bar{\boldsymbol{\theta}}\right) + \sum_{i=1}^{n} \boldsymbol{G}'\left(\boldsymbol{\theta} - \boldsymbol{\theta}_0\right)$$

in which $\bar{\boldsymbol{\theta}}$ is some vector between $\boldsymbol{\theta}$ and $\boldsymbol{\theta}_0$.

Asymptotic Distribution of GMM Estimators

- Now we substitute this mean value expansion into the first order condition.
- First order condition

$$\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{G}\left(\boldsymbol{w}_i,\boldsymbol{\theta}\right)\right]\widehat{\boldsymbol{\Xi}}\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\boldsymbol{\theta}\right)\right]=0$$

- Mean value expansion

$$\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\boldsymbol{\theta}\right)=\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)+\sum_{i=1}^{n}\boldsymbol{G}'\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right)$$

- Substitution

$$\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{G}\left(\boldsymbol{w}_i,\boldsymbol{\theta}\right)\right]\widehat{\boldsymbol{\Xi}}\left[n^{-1}\left(\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)+\sum_{i=1}^{n}\boldsymbol{G}'\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right)\right)\right]=0$$

Asymptotic Distribution of GMM Estimators

- Now replace random averages with their plims.
- So there should be an $o_p(1)$ term floating around.

$$\begin{aligned}\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{G}\left(\boldsymbol{w}_i,\boldsymbol{\theta}\right)\right]\widehat{\boldsymbol{\Xi}}\left[n^{-1}\left(\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)+\sum_{i=1}^{n}\boldsymbol{G}'\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right)\right)\right] &= 0\\ \boldsymbol{G}_0\boldsymbol{\Xi}_0\left[n^{-1}\left(\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)\right)+\boldsymbol{G}_0'\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right)\right] &= o_p(1)\end{aligned}$$

and solve away.

Asymptotic Distribution of GMM Estimators

- Solving away . . .

$$
\begin{aligned}
\boldsymbol{G}_0\boldsymbol{\Xi}_0\left[n^{-1}\left(\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)\right)+\boldsymbol{G}_0'\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right)\right] &= o_p(1)\\
\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right) &= -\boldsymbol{G}_0\boldsymbol{\Xi}_0\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)\right]+o_p(1)\\
\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right) &= -\left(\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\right)^{-1}\boldsymbol{G}_0\boldsymbol{\Xi}_0\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)\right]+o_p(1)\\
\sqrt{n}\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right) &= -\left(\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\right)^{-1}\boldsymbol{G}_0\boldsymbol{\Xi}_0\left[n^{-1/2}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)\right]+o_p(1)
\end{aligned}
$$

- The right-hand side of the second-to-last line contains the influence function.
- Look at what we have done: the argmin of a complicated function is now a **linear** function of the sample moments, up to $o_p(1)$. The influence function is the linearization of the first-order condition.

Asymptotic Distribution of GMM Estimators

- So the variance of the GMM estimator can be obtained by covarying the influence functions!

$$
\begin{aligned}
E\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right)\left(\boldsymbol{\theta}-\boldsymbol{\theta}_0\right)' &\equiv \\
E\left\{\left(\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\right)^{-1}\boldsymbol{G}_0\boldsymbol{\Xi}_0\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)\right]\left[n^{-1}\sum_{i=1}^{n}\boldsymbol{g}\left(\boldsymbol{w}_i,\bar{\boldsymbol{\theta}}\right)\right]'\boldsymbol{\Xi}_0\boldsymbol{G}_0'\left(\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\right)^{-1}\right\} &= \\
\left(\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\right)^{-1}\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{\Lambda}\boldsymbol{\Xi}_0\boldsymbol{G}_0'\left(\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\right)^{-1}&
\end{aligned}
$$

- Note that if we set $\boldsymbol{\Xi}_0 \equiv \boldsymbol{\Lambda}_0^{-1}$, then this mess reduces as follows:

$$
\begin{aligned}
\left(\boldsymbol{G}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{G}_0'\right)^{-1}\boldsymbol{G}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{\Lambda}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{G}_0'\left(\boldsymbol{G}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{G}_0'\right)^{-1} &= \\
\left(\boldsymbol{G}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{G}_0'\right)^{-1}\boldsymbol{G}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{G}_0'\left(\boldsymbol{G}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{G}_0'\right)^{-1} &= \\
\left(\boldsymbol{G}_0\boldsymbol{\Lambda}_0^{-1}\boldsymbol{G}_0'\right)^{-1}&
\end{aligned}
$$

What the derivation bought us

- Three objects, and the whole day is about producing them well:
  - $\widehat{\boldsymbol{\Lambda}}$: the covariance matrix of the moments. Invert it and you have the optimal weight matrix. Hour 2: stack influence functions.
  - $\widehat{\boldsymbol{G}}$: the Jacobian of the moments in the parameters. In SMM: numerical derivatives. Coming right up.
  - $\frac{1}{n}(\widehat{\boldsymbol{G}}\widehat{\boldsymbol{\Lambda}}^{-1}\widehat{\boldsymbol{G}}')^{-1}$: the covariance matrix of $\hat{\boldsymbol{\theta}}$.
- For a real, extremely brief, but logically similar derivation, see Newey and McFadden (1994).
- Now back to SMM: what changes when the moments come off a simulation?

Inference

- The simulated moments estimator is asymptotically normal for fixed S!! (This is not the case for simulated MLE.)
- The asymptotic distribution of $\hat{\theta}$ is given by

$$\sqrt{n}\left(\hat{\theta}-\theta\right) \xrightarrow{d} \mathcal{N}\left(0, \operatorname{avar}(\hat{\theta})\right)$$

in which

$$\operatorname{avar}(\hat{\theta}) \equiv \left(1+\frac{1}{S}\right)\left[\frac{\partial g_n(\theta)}{\partial \theta} W \frac{\partial g_n(\theta)}{\partial \theta'}\right]^{-1},$$

where W is the efficient weight matrix.

- Stare at the bracketed term: it is exactly the $\left(\boldsymbol{G}W\boldsymbol{G}'\right)^{-1}$ object we just derived for GMM. The **only** new ingredient is the $(1+1/S)$ factor.
- Note larger S (more simulated samples) $\Rightarrow$ smaller standard errors.

Where does the $(1 + 1/S)$ come from?

- Your moment condition contains **two** sources of noise:
  - Sampling error in the data moments: variance proportional to $1/n$
  - Simulation error in the simulated moments: variance proportional to $1/(Sn)$
- The two are independent, so the variances add:

$$\frac{1}{n} + \frac{1}{Sn} = \frac{1}{n}\left(1 + \frac{1}{S}\right)$$

- With $S = 10$, simulation noise costs you only 10% in variance. Cheap.
- This is also why you should not skimp on S: it is the one free lunch in this whole business.

The Jacobian (numerical derivative)

The avar formula needs $\boldsymbol{G} = \partial g_n(\theta)/\partial\theta$. In GMM you differentiate a formula. In SMM there is no formula. The moments come off a simulation, so you difference numerically:

1. Get the SMM estimate, $\hat{\theta}$
2. Remember the dimensions:
   - Parameters θ: $k \times 1$
   - Moments g: $m \times 1$
   - Jacobian $\partial g_n(\theta)/\partial\theta$: $k \times m$
3. Choose step size $h_i > 0$ for each parameter $i = 1, ..., k$
   - Each parameter gets its own step size
   - Must be chosen with care! Smaller is better ... except when it's not
   - Good initial guess: 1% of the parameter's estimate
4. Compute the two-sided difference. Element $\{i, j\}$ of $\partial g_n(\theta)/\partial\theta$ is

$$\frac{\partial g_{n,j}(\theta)}{\partial \theta_i} \approx \frac{g_{n,j}(\hat{\theta} + h_i) - g_{n,j}(\hat{\theta} - h_i)}{2h_i},$$

   where "$+h_i$" means perturb parameter i upward by h_i

Step sizes are an engineering problem, not a footnote

- Your objective function is computed off a simulation, so it is **noisy** at fine scales.
- Step size too small $\Rightarrow$ you are differentiating simulation noise. Garbage Jacobian.
- Step size too big $\Rightarrow$ truncation error. Also garbage.
- Plot $g_{n,j}(\hat{\theta} + h)$ against h for a range of h. You want the region where the implied slope is stable.
- This is a perfect AI task: "plot each moment against each parameter over a grid of step sizes." Five minutes of compute, and it will save you from a garbage covariance matrix.
- Remember this issue. It comes back with a vengeance in the identification diagnostics.

Test of overidentifying restrictions

- As in the case of plain vanilla GMM, one can perform a test of the overidentifying restrictions of the model:

$$\frac{nS}{1+S}Q(\hat{\theta}, n)$$

- This statistic converges in distribution to a χ^2 with degrees of freedom equal to the dimension of g_n minus the dimension of θ.

- You can also put a t-stat on each **individual** moment condition. For that you need the covariance matrix of the moment errors:

$$\text{var}(g_n(\hat{\theta})) = \frac{1}{n}\left(1+\frac{1}{S}\right)\left(I - \boldsymbol{G}'(\boldsymbol{G}W\boldsymbol{G}')^{-1}\boldsymbol{G}W\right)\hat{\boldsymbol{\Lambda}}\left(I - \boldsymbol{G}'(\boldsymbol{G}W\boldsymbol{G}')^{-1}\boldsymbol{G}W\right)'.$$

- In practice you will almost always reject the χ^2 test. Your model is a deliberate simplification. The interesting question is **which** moments the model misses, not **whether** it misses .

One slide on indirect inference

- Gourieroux, Monfort and Renault (1993): instead of matching moments, match the coefficients of an **auxiliary model** estimated on real and simulated data.
- Regression slopes, VAR coefficients, probit coefficients, . . .
- Mechanically, an auxiliary-model coefficient *is* a function of moments: it is a statistic you compute identically on both datasets.
- So the objective function, the weight matrix, the influence functions, the engineering are all essentially the same.

Outline

Copyright ©Toni M. Whited

The weight matrix

- In most applications, one can use the **optimal** weight matrix, which we will call $\mathbf{\Lambda}^{-1}$, in which $\mathbf{\Lambda}$ is the variance-covariance matrix of $n^{-1}\sum_{i=1}^{n} g\left(\boldsymbol{w}_i\right)$.
- **This weight matrix can be calculated without any knowledge of the model!!!**
- It is a property of the *data*, not of the simulation. You compute it once, before you ever solve the model.
- Either use GMM or stack influence functions.
  - Computationally identical.
  - We do influence functions, because they scale to any pile of statistics you can dream up.

First: the parade of bad weight matrices

- Many structural estimation papers use odd weighting matrices.

- **Identity matrix**
  - Mechanically puts more weight on moments that are larger in absolute value.
  - Not an economically sensible choice. Bazdresch, Kahn and Whited (2018) note poor finite sample performance.

- **Inverse squared moments on the diagonal**
  - Minimizes percentage differences, so it puts more weight on moments that are small in absolute value.
  - Equally arbitrary, in the opposite direction.

- **Inverse moment variances on the diagonal**
  - Downweights imprecise moments. More or less statistically sensible.
  - But it ignores the covariances, and the covariances matter.

Copyright ©Toni M. Whited

The right weight matrix

- **Inverse clustered moment covariance matrix**
  - This choice minimizes overall model error.
  - Takes into account moment covariances.
  - Bazdresch et al. (2018) note good finite sample performance.
- If at all possible, do this.
- The worry that optimal weighting has terrible finite-sample properties is a misreading of Altonji and Segal (1996), which considers a problem in which the parameters show up in the weight matrix. That **does not happen** in SMM: the weight matrix comes from the data alone.

Why am I torturing you with influence functions?

- Your moment vector will be a grab bag: means, variances, autocorrelations, regression slopes, maybe from different samples at different frequencies.
  - How do you estimate the covariance between a mean and a variance?
  - Between slopes from two separate regressions?
  - Especially if you need to cluster by firm?
- You need the **joint** covariance matrix of that entire grab bag.
- Delta-method algebra for a 15-moment vector is a nightmare of chain rules.
- Bootstrapping means recomputing everything hundreds of times, and it has worse finite-sample properties than what I am about to show you.
- Influence functions turn the whole problem into: build one $n \times m$ matrix, then take an inner product.
- Beyond structural estimation: if you know how to use influence functions, **you know how to estimate the standard error for anything!**

Copyright ©Toni M. Whited

Motivating example

- Let $w_1, \ldots, w_n$ be an $i.i.d.$ sample of a scalar random variable. We want to estimate the covariance between the sample mean and the sample variance. There is an established GMM theory to do this, but let's try a different way.
- Rewrite the sample mean as

$$\bar{w} = \frac{1}{n} \sum_{i=1}^{n} w_i$$

- And the sample variance as

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^{n} (w_i - \bar{w})^2$$

Copyright ©Toni M. Whited

Now let's do the covarying

Therefore,

$$\begin{aligned}
\text{cov}(\bar{w}, \hat{\sigma}^2) &= \frac{1}{n^2}\text{cov}\left(\sum_{i=1}^{n} w_i, \sum_{i=1}^{n}(w_i - \bar{w})^2\right) \\
&= \frac{1}{n^2}\sum_{i=1}^{n}\text{cov}\left(w_i, (w_i - \bar{w})^2\right) \quad \text{(By independence)} \\
&= \frac{1}{n^2}\sum_{i=1}^{n}\mathbb{E}\left[(w_i - \bar{w})\left((w_i - \mu)^2 - \sigma^2\right)\right] \quad \text{(By covariance definition)} \\
&\approx \frac{1}{n^2}\mathbb{E}\left[(w_i - \bar{w})\left((w_i - \bar{w})^2 - \hat{\sigma}^2\right)\right] \quad \text{(By Slutsky)}
\end{aligned}$$

Motivating example

- To estimate the covariance in the last term, use the analogy principle.
- Using a data series, w_i, calculate the series $(w_i - \bar{w})$ and $((w_i - \bar{w})^2 - \hat{\sigma}^2)$.
- Take the sample average of the product, which is consistent for the population expectation above.
- Divide by another n, and then you're done.
- The terms $(w_i - \bar{w})$ and $((w_i - \bar{w})^2 - \hat{\sigma}^2)$ are examples of influence functions.

Copyright ©Toni M. Whited

Motivating example

- The magic is that we've turned the "hard" problem of calculating a covariance into the "easy" problem of writing each estimator as a sample mean.
- We then can use the analogy principle to estimate the covariance between the two estimators.
- In this simple example, the trick seems trivial, but imagine if you had a lot of different estimators: sample means, sample variances, regression slopes, conditional skewness, etc.

Copyright ©Toni M. Whited

The general definition of an influence function

- Consider any estimator $\hat{\theta}$, and suppose there is a function $\phi(\boldsymbol{w}_i)$ such that

$$\sqrt{n}\left(\hat{\theta} - \theta_0\right) = \frac{1}{\sqrt{n}} \sum_{i=1}^{n} \phi(\boldsymbol{w}_i) + o_p(1), \qquad E(\phi(\boldsymbol{w}_i)) = 0$$

- Then $\phi(\boldsymbol{w}_i)$ is the influence function of $\hat{\theta}$: observation i's "contribution" to the estimate.

- The estimator is approximately an average of influence-function values, so the variance of the influence function is the asymptotic variance of the estimator.

- You have already met one! In Hour 1, linearizing the GMM first-order condition gave

$$\phi(\boldsymbol{w}_i) = -\left(\boldsymbol{G}_0 \boldsymbol{\Xi}_0 \boldsymbol{G}_0'\right)^{-1} \boldsymbol{G}_0 \boldsymbol{\Xi}_0 \, \boldsymbol{g}\left(\boldsymbol{w}_i, \hat{\boldsymbol{\theta}}\right)$$

- Nearly every statistic you would ever match is a GMM estimator: means, variances, conditional skewness, regression coefficients.

Copyright ©Toni M. Whited

Example: the mean

- What is the influence function for the estimate of the mean of a random variable z_i?

$$\begin{aligned} \boldsymbol{g}\left(\boldsymbol{w}_i, \hat{\boldsymbol{\theta}}\right) &\equiv z_i - \hat{\mu} \\ \boldsymbol{G}_0 &\equiv -1 \\ \phi\left(\boldsymbol{w}_i\right) &\equiv z_i - \hat{\mu} \end{aligned}$$

- In words: observation i pushes the sample mean around in proportion to its own deviation from the mean. Sensible.

- The sample variance of $(z_i - \hat{\mu})$, divided by n, is the variance of $\hat{\mu}$. You have known this since your first econometrics course. Now it has a fancy name.

Example

- Recall the definition of an influence function :

$$-\left(\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{G}_0'\right)^{-1}\boldsymbol{G}_0\boldsymbol{\Xi}_0\boldsymbol{g}\left(\boldsymbol{w}_i,\hat{\boldsymbol{\theta}}\right)$$

- Consider a simple linear regression

$$y_i = x_i\beta + u_i$$

- What is the influence function for $\hat{\beta}$?

$$\begin{aligned}
\boldsymbol{g}\left(\boldsymbol{w}_i,\hat{\boldsymbol{\theta}}\right) &\equiv x_i\cdot\hat{u}_i = x_i\cdot\left(y_i - x_i\hat{\beta}\right)\\
\boldsymbol{\Xi}_0 = \boldsymbol{\Lambda}_0^{-1} &\equiv \sigma^{-2}E\left(x_i'x_i\right)^{-1}\\
\boldsymbol{G}_0 &\equiv -E\left(x_i'x_i\right)\\
\phi\left(\boldsymbol{w}_i,\hat{\boldsymbol{\theta}}\right) &\equiv E\left(x_i'x_i\right)^{-1}\left(x_i\cdot\left(y_i - x_i\hat{\beta}\right)\right)
\end{aligned}$$

Which can be written as:

$$\left(N^{-1}\sum_{i=1}^{N}\left(x_i'x_i\right)\right)^{-1}\left(x_i\cdot\hat{u}_i\right)'$$

where the operator $\cdot$ is the Hadamard element-by-element operator.

Example: a regression slope

- Stare at this. It is the summand in the sandwich formula for heteroskedasticity-robust standard errors.

$$\phi\left(\boldsymbol{w}_i\right) \quad \equiv \quad \left(n^{-1}\sum_{i=1}^{n} x_i' x_i\right)^{-1} \left(x_i \cdot \hat{u}_i\right)$$

- Influence functions are not exotic. They are the robust standard errors you already trust, reorganized so that they **stack**.

Stacking

- Suppose your moment vector contains the mean μ of one variable and the OLS slope β from a regression involving others.
- You need the covariance between $\hat{\mu}$ and $\hat{\beta}$, because they are computed from the same firms.
- Let $\hat{\phi}_\mu$ be the $n \times 1$ sample influence function for μ, and $\hat{\phi}_\beta$ the $n \times k$ sample influence function for β. Define

$$\Phi \equiv \left[\begin{array}{cc} \hat{\phi}_\mu & \hat{\phi}_\beta \end{array} \right] \qquad (n \times (k+1))$$

- Then the covariance matrix of $(\hat{\mu}, \hat{\beta})$ is

$$\hat{\mathbf{\Lambda}} = \Phi' \Phi \, n^{-2}$$

- That's it. One column per moment, one row per observation, one inner product.[1]

[1] The reference is Erickson and Whited (2002).

Copyright ©Toni M. Whited

Stacking, in pictures

- With m moments, Φ is $n \times m$, and $\hat{\mathbf{\Lambda}} = \Phi'\Phi n^{-2}$ is $m \times m$.
- Here is what the stack looks like in a spreadsheet, for a mean plus a regression with four regressors:

	A	B	C	D	E
1	ϕ_μ	$\phi_{\beta 1}$	$\phi_{\beta 2}$	$\phi_{\beta 3}$	$\phi_{\beta 4}$
2	-0.3077937	-0.2881243	-0.1493863	-0.4944986	0.09112854
3	-0.118798	0.35481714	-0.4654646	0.4161474	-0.2297822
4	0.27052049	-0.0679981	-0.2685441	-0.0713374	-0.2145772
5	0.23215481	-0.4571358	-0.3404458	0.00301565	0.20407327
6	0.19164567	0.19907597	0.49640239	-0.4696586	0.02729855
7	-0.4222975	-0.2423789	-0.477901	-0.1465544	-0.2059395
8	-0.0261526	-0.2232136	-0.275062	0.43224294	0.08048878
9	-0.2780236	-0.0397985	0.4320227	-0.2087948	-0.3908644
10	-0.2101804	0.44238463	0.371486	-0.1105543	-0.1978471
11	-0.0332542	0.38275707	0.31075324	-0.2856035	0.40799314
12	-0.2346248	0.21748158	0.19450942	-0.1521142	-0.253551
13	0.4381279	-0.1185061	0.04483009	-0.0954184	-0.4959698
14	-0.4511052	0.09310356	-0.1128563	-0.1910718	0.12800501
15	0.08186761	-0.2591236	0.21970691	0.09055461	-0.3267252
16	-0.4999294	-0.058372	-0.2427571	0.02993619	0.29671955
17	-0.2666323	0.44120731	0.15008241	-0.0226203	0.47779926

- Your weight matrix is $\hat{\mathbf{\Lambda}}^{-1}$. Done. No model required.

Sample Julia code

```
# Mean influence function
n = size(z,1);
meaninflnc = z .- mean(z);

# OLS influence function
bhat = inv(x'*x)*x'*y;
uhat = y - x*bhat;
olsinflnc = (inv((x'*x)./n) * ((x.*uhat)'))';

#Big influence function
biginflnc = zeros(size(x,1),size(x,2)+1);
biginflnc[:,1] = meaninflnc;
biginflnc[:,2:size(x,2)+1] = olsinflnc;

#Covary the influence functions
avar = biginflnc'*biginflnc ./(n^2);
```

Clustering: because your data are not $i.i.d.$

- Everything so far is for $i.i.d.$ data. Data are almost never $i.i.d.$ in corporate finance or any other applied micro field.
- Suppose the sample consists of K clusters (e.g. firms), independent across clusters but dependent within.
- Recipe: **sum the influence-function rows within each cluster** before taking the inner product.
- Let $\tilde{\boldsymbol{\phi}}_k = \sum_{j \in k} \hat{\boldsymbol{\phi}}_j$, the $1 \times m$ within-cluster sum. Then

$$\hat{\boldsymbol{\Lambda}} = \frac{1}{n^2} \sum_{k=1}^{K} \tilde{\boldsymbol{\phi}}_k' \tilde{\boldsymbol{\phi}}_k$$

- Cluster on firm if shocks persist within firm; cluster on time if shocks are common across firms. If both scare you, look up two-way clustering, but at minimum do one of them.

Moments from different samples, frequencies, or geographies

- Some moments from Compustat, some from the PSID? If the samples are **independent**, the influence-function stack is block diagonal:

$$\hat{\mathbf{\Lambda}} = \begin{bmatrix} \hat{\mathbf{\Lambda}}_1 & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{\Lambda}}_2 \end{bmatrix}$$

  Just be careful about which "n" divides which block.

- Moments at different frequencies, or with partially overlapping samples? Stack influence functions, insert zeros in the spots for non-overlapping observations, and mind the n's (Erickson and Whited 2012).

- Aggregate time-series moments plus panel moments over similar periods? Stack and cluster by time period.

- None of this is deep. It is bookkeeping. Which is exactly why you can delegate the code, but **not** the bookkeeping decisions, to an AI.

Copyright ©Toni M. Whited

What if you truly cannot get the covariances?

- Sometimes you match moments lifted from other papers, and all you have is their standard errors.
- Cocci and Plagborg-Møller (2024) derive worst-case standard errors that are valid using only the *diagonal* of the moment covariance matrix.
- **Clever** and useful **as a last resort**.
- Most of the motivating cases in that paper are actually solvable by stacking influence functions with a little care. Try the stack first.

Outline

Copyright ©Toni M. Whited

We will use the formal statistical definition of identification

- Econometrician defines an objective function over parameters and data
- Goal: Select parameter values that minimize this objective function
- A parameter is (point) *identified* if there is a unique minimum for the objective function at the parameter's true value in the population
- "Identification" means something different in reduced-form work (an experimental design that establishes causality). Sigh. Do not confuse the two in your talks, and do not let your discussants confuse them either.

“Identification is not causality, and vice versa”

- (Kahn and Whited 2018), Review of Corporate Finance Studies
- Exogenous variation is:
  - Always necessary to identify a causal relation
  - Never sufficient for identifying an economically interesting parameter
    - You also need an economic model (either mathematical or verbal)
  - Only sometimes necessary to identify an economically interesting parameter
- Interesting parameters can sometimes be identified without exogenous variation
  - This is often what’s going on in structural corporate finance
- Identification **always** leans on assumptions, even in reduced-form papers with exogenous variation. Those papers just lean on a “verbal” economic model.

Identification and SMM

- The success of SMM relies on picking moments g that can identify the structural parameters θ.
- Global identification: the expected difference between simulated and actual moments equals zero iff the structural parameters are at their true values.
- Local identification: the Jacobian

$$\partial g\left(y_{is}\left(\theta\right)\right)/\partial\theta$$

  has full rank.
- Loose interpretation: the moments are **informative** about the parameters. In other words, the sensitivity of moments to parameters is high.
- It's usually impossible to prove global identification for a model without closed-form moments. So we gather evidence like detectives instead.

How to choose moments

- Best-case scenario: Each moment depends on just one model parameter
  - "Moment #1 identifies parameter #1, moment #2 identifies parameter #2..."
  - This is how many macro papers are (misleadingly) written
  - If it were literally true, you would not need SMM at all
- More realistic: Every moment depends on every parameter
- All parameters can affect all moments, but the mapping has to be one-to-one and onto
- Do comparative statics to understand how each moment moves with each parameter. **Make sure you understand the economics behind each comparative static.**
- Need enough moments, and moments that move in different directions for different parameters, to obtain identification
- Calculate many more moments than you think you will need. You **will** change your moment vector at some point.

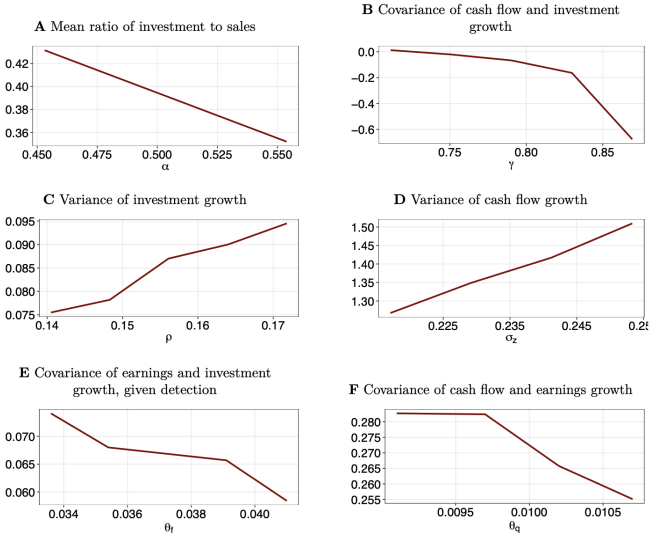
Copyright ©Toni M. Whited

Use economics!

- **PLAY WITH YOUR MODEL UNTIL YOU UNDERSTAND HOW IT WORKS!!!!!!!!!**
- Do comparative statics: plot the simulated moments as functions of the parameters.
- You want to find steep, monotonic relationships.
- You want moments that move in different directions for different parameters.
- Perfect AI task. Ask it to sweep every parameter and plot every moment overnight. **There is no longer any excuse for skipping this step.**
- But the AI cannot tell you *why*, without hallucinating, the exit rate rises with the fixed cost. If you cannot explain the economics of each plot, you do not understand your own estimator.

Copyright ©Toni M. Whited

Example One: Terry, Whited and Zakolyukina (2023)



Identification checks

- Check #1: Check the standard errors
  - Huge standard errors are usually a symptom of poor identification
  - The standard errors depend on $\partial g\left(y_{is}\left(\theta\right)\right)/\partial\theta$, the sensitivity of moments to parameters
  - If the sensitivity is low, the derivative will be near zero, which produces huge standard errors for the structural parameters
- Check #2: Does the Jacobian have full rank?
  - If not, your model is not locally identified
  - Almost the same as the standard error check, but not quite

Computer output from an estimation gone wrong

```
  Parameter estimates:
&        0.100 &        0.231 &        0.490 &        0.756 &        0.518 &        0.128 &        0.000 &        0.008 &
&(       2.033)&(       0.013)&(       0.000)&(       0.073)&(       0.264)&(       0.000)&(       0.004)&(       0.000)&(

  Gradient matrix:

       0.00000000       0.00000000       1.02758239       0.00000000       0.00000000      38.25073505       0.00000000       0.00000000
       0.00000000       0.00000000       0.28512603       0.00000000       0.00000000       4.82100673       0.00000000       0.00000000
       0.00000000       0.00000000      -9.97948112       0.00000000       0.00000000     370.80545008       0.00000000       0.00000000
       0.00000000       0.00000000       0.40639694       0.00000000       0.00000000       1.02101543       0.00000000       0.00000000
       0.00000000       0.00000000       0.84485716       0.00000000       0.00000000       0.04210265       0.00000000       0.00000000
       0.00000000       0.00000000    -138.58951690       0.00000000       0.00000000       0.00889156      -0.00000004       0.00000000
       0.00000000       0.05056345       0.03244400      -0.16356671       0.00000000      -5.85773601      -2.79010277      -2.79010369
       0.00000000       0.01667884       0.00342592      -0.00333117       0.00000000      -0.09697136      -0.07479289      -0.07479291
       0.00000000      -0.29607166       1.09787363       0.14226033       0.00000000     111.43914416       0.00006803       0.00000173
      -2.06371354       3.34618706      10.68850788     -11.02873288      -0.77796138     -74.85216998     736.96355852    -153.90999884
      -0.01039442       0.02335486      -0.62675135      -0.05792594      -0.00419307      -6.10626047       3.66442260      -0.78400489
       0.00000000       0.04861350      -3.09825605       0.11134717       0.00898020     -25.21239839       2.03722313       1.89152864
       0.00000000       0.00122617      -0.74021211      -0.00036989       0.00009098      -0.66126219      -0.00761401      -0.00819525
       0.00000000       0.08164404      -1.21010451      -0.13848512      -0.00552660      -3.77971077      -2.64720025      -2.53142955
       0.00000000       0.00444937       0.02217510      -0.00057544       0.00000972      -0.26535678      -0.01661087      -0.01716714
       0.00000000       0.00038676       0.00024816      -0.00125113       0.00000000      -0.04480597      -0.02134156       0.11961819
```

Local identification diagnostic

- Andrews, Gentzkow and Shapiro (2017) provide a local diagnostic that measures the sensitivity of $\hat{\boldsymbol{\theta}}$ to the estimated moments, $\hat{\boldsymbol{g}}$.
- Let $\boldsymbol{G}$ be the Jacobian of g w.r.t θ.
- The diagnostic is (and it should look familiar because it is the GMM influence function skeleton):

$$\boldsymbol{S} = -(\boldsymbol{G}'\boldsymbol{\Xi}\boldsymbol{G})^{-1}\boldsymbol{G}'\boldsymbol{\Xi}.$$

- This measure is not scale invariant, so normalize:

$$\boldsymbol{S}_{l,p}\sqrt{\boldsymbol{\Lambda}_{l,l}}$$

  where $\boldsymbol{\Lambda}_{l,l}$ is a moment variance.
- This diagnostic captures both
  - the steepness of the gradients
  - the precision of the estimation of the moments that identify the parameters
- Warning from experience: it is **very** sensitive to the step size in the numerical gradient. (Told you the step sizes would come back.)

Copyright ©Toni M. Whited

Outline

Copyright ©Toni M. Whited

Now, the mess

- On paper, SMM is a quadratic form and an inner product. Two slides of econometrics.
- In practice, it is a small software engineering project:
  - a data pipeline, a model solver, a simulator, a moment calculator, an optimizer, and an inference module
- Six components, and **every single one can fail silently**.
- A silent failure does not crash. It produces plausible-looking parameter estimates that are **wrong**.
- Everything I used to present as “miscellaneous tips from the trenches” is really a systematic answer to one question: *how do you engineer this project so that silent failures get caught?*

The architecture

- An optimizer sits on top. It has two inputs: a parameter vector and a function to optimize.
- The function to be optimized eats parameters and spits back the objective:
  1. read data moments and weight matrix (computed **once**, offline)
  2. solve the model at θ
  3. simulate the model
  4. compute simulated moments
  5. form the quadratic form
- Keep the links in this chain **separate**: separate functions, separate tests.
- AI-era corollary: never ask an AI to “write me an SMM estimator.” Ask for one link at a time, with a test for each. The chain is only as trustworthy as its least-tested link.

Copyright ©Toni M. Whited

Pseudo code

```
function SMM_Objective_Function(in double parameters[numberParameters], out double objectiveFunctionValue)
        read weightMatrix[numberMoments, numberMoments]
        read dataMoments[numberMoments]
        call function solveTheModel( in double parameters[numberParameters],
                out double valueFunction[stateSpaceSize],
                out int policyFunction[stateSpaceSize])
        call function simulateFirms( in double valueFunction[stateSpaceSize],
                in int policyFunction[stateSpaceSize],
                out double simulatedFirms[numberOfFirms, numberOfVariables])
        call function calculateMoments(in double simulatedFirms[numberOfFirms, numberOfVariables],
                out double SimulatedMoments[numberMoments])
        momentError = dataMoments − SimulatedMoments
        objectiveFunctionValue = momentError′ * weightMatrix * momentError
```

The data step

- Choose moments to match: means, variances, covariances, regression slopes, empirical policy functions. Need at least as many moments as parameters.
- Compute the moments in the actual data and stack them in a vector.
- Estimate the covariance matrix of this vector by stacking influence functions. Invert it. This is your weight matrix.
- Calculate many more moments than you think you will need. Referees will ask; you **will** change the moment vector.
- Do not bootstrap. You now know an easier and asymptotically more efficient way that has better finite-sample properties.

Copyright ©Toni M. Whited

The model step: do not construct a black box

- More parameters $\neq$ a better model!!!!!!
- Different features of the data should change when underlying parameters change.
- This is less likely to happen in a model with many extraneous parameters.
- If the author cannot clearly explain which features of the data identify each parameter, the paper / job market candidate is a “reject.”
- AI makes this failure mode **more** tempting, not less: adding a feature to the model now costs one prompt. Resist. Model complexity is still expensive. You just pay in identification and interpretability instead of coding time.

Copyright ©Toni M. Whited

How not to construct a black box

- Start simple!!!
- Make sure the simple model is **right**.
- Figure out where the simple model succeeds and fails.
- Add a feature that will help you answer the question you want to answer.
- Make sure the slightly more complicated model is right.
- Figure out where the more complicated model succeeds and fails.
- Converge.
- The preceding is eerily the same as, but not taken from, this video by Michael Keane: `https://www.youtube.com/watch?v=0hazaPBAYWE`. If you don't believe me, maybe you will believe him.

Copyright ©Toni M. Whited

The question comes first. Not the model

- Before going structural, convince yourself that a structural approach is absolutely necessary.
- Are there data limitations?
- Are you interested in something besides a causal elasticity?
- An AI will happily build you a 12-parameter model of anything. It will never ask you *why*. That question is yours.

Copyright ©Toni M. Whited

The simulation step: random numbers

- Computers produce “pseudo-random” numbers: deterministic sequences, e.g.

$$X_r = (kX_{r-1} + c) \bmod m,$$

  that pass simple tests of randomness. The seed X_0 determines the entire sequence.

- **Use the same seed, hence the same draws, every time you evaluate the objective function. Do not mess up this step.**

- If the draws change across evaluations, the objective function jitters, and your optimizer chases simulation noise instead of θ.

- Best practice: draw the shocks **once**, up front, and pass the same array into every simulation.

- I have seen AI-generated code re-seed inside the simulation loop, seed from the clock, and use thread-dependent random streams. All three are fatal, none of them crash, and all of them produce plausible-looking output. **Check the seeding yourself.**

Copyright ©Toni M. Whited

The optimizer step

- You are going to have to minimize an objective function with no closed form, no reliable gradient, and possibly many local minima.
- You can't use a gradient-based method unless you have a closed-form GMM or MLE problem.
- Multi-start algorithms combined with Nelder-Mead tend to work very poorly, except, maybe:
- The Tiktak algorithm (Arnoud, Guvenen and Kleineberg 2017), but the original uses Dropbox for parallelization . . . I have a Julia version that is better.
- Use simulated annealing (SA), particle swarm (PS), or differential evolution (DE) to avoid local minima.
- DE and PS can be parallelized, but they can converge way too fast or not at all, depending on technical parameter settings. In technical terms, they're downright squirrelly.
- So: do not just accept the default settings. Do not let an AI pick them for you either, because it will just parrot the defaults back at you.

Copyright ©Toni M. Whited

Did you actually find the global minimum?

- No optimizer comes with a certificate. You have to check.
- Check #1: Start searching from different initial parameter guesses
  - Should reach the same final estimate regardless of the initial guess
  - If not: Coding errors? Stuck in a local min? Model not identified?
- Check #2: Babysit your code
  - Some parameters converge faster than others. Keep track of this.
  - If one or two parameters are bouncing around drastically during the estimation, you have a problem.

Copyright ©Toni M. Whited

Do not treat the process of estimating the model mechanically!

- First, try to calibrate the model by hand.
- You will learn about which parts of the model are useful for understanding which parts of the data.
- You will learn a lot about the economics of the model.
- You will also learn a lot about identification.
- You will end up with a good starting value for your estimation.
- Only after you have done this should you start your estimation running.
- This advice has aged **beautifully**. Hand-calibration is the one step where you, the economist, stare at the mapping from parameters to moments. The optimizer (and the AI) can only automate what you already understand.

Copyright ©Toni M. Whited

The speed problem

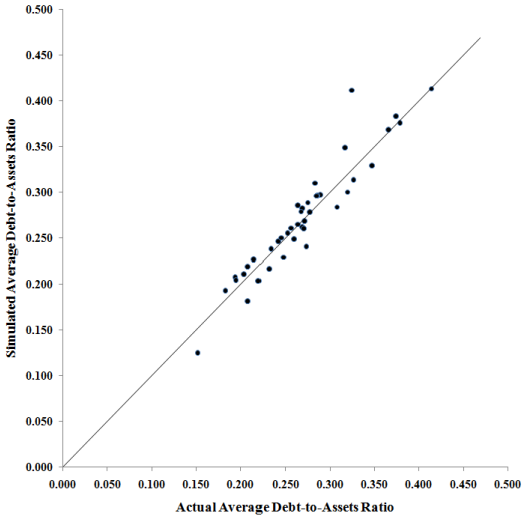
- To estimate a model, you usually have to solve it $\sim$50,000 times.
- A model solution that takes one minute to solve means a many-day estimation. Per run. And you will do many runs.
- So: do not use a slow interpreted language for the inner loop. Use a compiled language: C, C++, Fortran, or (in my experience, not badly) Julia.
- The AI-era update: **the cost of writing fast code has collapsed.**
  - AI is genuinely good at translating a working prototype into Fortran/C/Julia
  - It is genuinely good at parallelization and GPU boilerplate
  - "I only know Matlab" is no longer a reason to run a many-day estimation
- But translation is a **verification** problem: the translated solver must produce the same value function as the prototype to machine precision. Test that. Do not assume it.

Copyright ©Toni M. Whited

The data step: agent heterogeneity

- It is really hard to address the issue of agent heterogeneity using SMM.
- This is perhaps the biggest drawback of this technique.
- There is no choice-specific error term as in many discrete choice techniques.
- The models we simulate are at best an equilibrium with very limited heterogeneity.
- So you have to suck as much heterogeneity out of your data as you can before you can have any hope of fitting the model to the data.
  - Firm and time fixed effects
- **And then do the exact same de-meaning to your simulated data.** This is the single most common way to violate the "same calculations on both datasets" rule.
- You can also do sample splits. This used to be computationally infeasible, but . . . computing is cheap now

This is from 2009![2]



[2] DeAngelo, DeAngelo and Whited (2011)

Copyright ©Toni M. Whited

Outline

Copyright ©Toni M. Whited

AI-generated code almost always **runs**

- Running is not the standard. **Right** is the standard.
- In most software, bugs announce themselves: the app crashes, the page is blank.
- In estimation, bugs produce $\hat{\theta} = 0.43$ instead of $\hat{\theta} = 0.61$, and both numbers look fine in a table.
- Nobody can look at 0.43 and see the bug.
- So the only defense is a testing regime designed around silent failure.

Copyright ©Toni M. Whited

AI is terrible at diagnosing failure

- AI will happily give you a converged solution.
- It will not tell you that your moment 3 (the equity premium) is pulling your parameter vector into a ludicrous region because your model lacks a disaster channel.
- It will not tell you that moment 7 (the payout ratio) is messing up the variance of your driving process.
- Many examples.

Copyright ©Toni M. Whited

Where AI code goes silently wrong: a field guide

- Timing conventions: is capital chosen at t productive at t or at $t+1$? The AI will pick one convention in the solver and the other in the simulator, and both files look correct alone.
- Moment definitions that differ subtly between the data code and the simulation code (levels vs. logs, gross vs. net, before vs. after de-meaning).
- Burn-in: forgetting to discard the first, possibily not-optimally generated part of a simulated panel.
- Clustering on the wrong dimension, or clustering the data moments but not thinking about the time-series structure of the simulation.
- Re-seeding in the wrong place (previous section).
- Every one of these produces plausible tables. AI just produces them faster.

Copyright ©Toni M. Whited

The test battery, link by link

- **Solver**: kill the frictions. Set adjustment costs to zero, or taxes to zero, and compare to the closed-form solution that usually exists in the frictionless case. Machine precision or bust.
- **Simulator**: simulate a giant panel and check that simulated policies obey the policy function.
- **Moment calculator**: one function, two datasets. Run it on the real data; run it on the simulation. If you have two separate moment routines, you very likely have a bug.
- **Weight matrix**: check the influence-function code against closed forms (the mean, the OLS sandwich). Check that $\hat{\Lambda}$ is symmetric positive definite *before* inverting.
- **Inference**: the Jacobian step-size plots from Hour 1.
- Make the AI write all of these tests. It is spectacular at writing tests. It just will not insist on them unless you do.

Copyright ©Toni M. Whited

The end-to-end test: Monte Carlo on fake data

- Simulate a "fake" dataset from the model at known parameters θ_0.
- Estimate the model, treating the fake data as if they were real. Full pipeline: moments, weight matrix, optimizer, standard errors.
- Does the estimator recover θ_0?
- Repeat M times: the standard deviation of the estimates across replications should match the average of your standard errors. If it does not, your inference code is wrong.
- This is the single most valuable test in structural estimation, and it used to be a luxury. With cheap compute and AI-written harness code, it is now **mandatory**. I will not believe your estimates without it, and neither should you.

Babysit your estimation

- Log **everything**: every parameter vector the optimizer tries, every objective value, every simulated moment vector.
- Watch the logs. Some parameters converge fast; some bounce around. If one parameter is drifting to a bound, you are learning something about identification, about your bounds, or about a bug.
- An estimation that runs for two days unwatched is two wasted days ninety percent of the time.
- This is another good AI delegation: have it write a monitor that plots the optimizer's path and flags stuck or runaway parameters. Then actually look at the plots.
- Remember: the optimizer will always return *some* number. Convergence of the algorithm is not convergence to the truth.

Copyright ©Toni M. Whited

Remember that you are actually doing estimation

- Get the standard errors right.
- The actual data are usually not $i.i.d.$
- When estimating the covariance matrix for empirical moments, you must take into account
  - Heteroskedasticity
  - Time-series autocorrelation
  - Cross-sectional correlation
  - Serial correlation, including correlation across moments
- You know what to do: stack the influence functions and cluster.
- A paper with beautiful economics and broken standard errors is not a beautiful paper.

Copyright ©Toni M. Whited

Working with an AI on all of this: my actual advice

- Write a short, human-readable spec first: variable definitions, timing conventions, moment definitions, sample filters. Feed it to the AI with every request. Most silent failures are two files disagreeing about a convention that lives only in your head.
- Small, verifiable pieces. One link of the chain per request, with tests.
- Make it explain the code back to you. If the explanation surprises you, one of you is confused, and it matters enormously which one.
- Keep everything in version control, and make the whole pipeline reproducible with one command. When (not if) you find a bug at the end, you will need to know exactly what changed and rerun everything.
- Never paste in a number it computed without knowing how you would check it.
- The AI does not have skin in the game. Your name is on the paper. **Verification is not part of the job. It is the job.**

Copyright ©Toni M. Whited

Outline

Copyright ©Toni M. Whited

The preflight checklist

1. One moment function, two datasets. Same fixed effects, same trimming, same everything.
2. Weight matrix: inverse **clustered** moment covariance, from stacked influence functions, verified against closed forms.
3. Comparative statics plotted for every (parameter, moment) pair, with the economics of each explained in words.
4. Frictionless special cases match closed forms to machine precision.
5. Shocks drawn once; same seed every objective evaluation.
6. Global optimizer, multiple starting values, same answer from each.
7. Monte Carlo on fake data: parameters recovered, standard-error coverage checked.
8. Jacobian step sizes chosen from stability plots, not defaults.
9. $(1 + 1/S)$ in the standard errors; overidentification statistic reported.
10. You can explain, without notes, which features of the data identify each parameter.

If you cannot check every box, you are not done. The AI can help you check boxes 1–9 faster than any RA in history. Box 10 is, and will remain, yours.

Copyright ©Toni M. Whited

Conclusion: SMM in the AI era

- The econometrics of SMM fits on two slides: a quadratic form, and an inner product of stacked influence functions.
- The hard part was never the econometrics. It is the engineering: a chain of components that fail silently, wrapped around an optimizer that always returns a number.
- AI has made the *coding* nearly free. It has made the *verification* more important than ever, because you will now run code you did not write.
- Your comparative advantage has not changed. It is economics: choosing the question, choosing the moments, and understanding the mapping between them.
- Fifteen years ago the constraint was “can you code it?” Now the constraint is “do you understand it?” I consider this an improvement.

Copyright ©Toni M. Whited

Altonji, J.G., Segal, L.M., 1996. Small-sample bias in gmm estimation of covariance structures. Journal of Business and Economic Statistics 14, 353–366.

Andrews, I., Gentzkow, M., Shapiro, J.M., 2017. Measuring the sensitivity of parameter estimates to sample statistics. Quarterly Journal of Economics 132, 1553–1592.

Arnoud, A., Guvenen, F., Kleineberg, T., 2017. Benchmarking global optimization algorithms. Manuscript, Yale University.

Bazdresch, S., Kahn, R.J., Whited, T.M., 2018. Estimating and testing dynamic corporate finance models. Review of Financial Studies 31, 322–361.

Cocci, M.D., Plagborg-Møller, M., 2024. Standard errors for calibrated parameters. Review of Economic Studies, forthcoming.

DeAngelo, H., DeAngelo, L., Whited, T.M., 2011. Capital structure dynamics and transitory debt. Journal of Financial Economics 99, 235–261.

Duffie, D., Singleton, K.J., 1993. Simulated moments estimation of Markov models of asset prices. Econometrica 61, 929–52.

Erickson, T., Whited, T.M., 2002. Two-step GMM estimation of the errors-in-variables model using high-order moments. Econometric Theory 18, 776–799.

Erickson, T., Whited, T.M., 2012. Treating measurement error in Tobin's *q*. Review of Financial Studies 25, 1286–1329.

Gourieroux, C.S., Monfort, A., Renault, E., 1993. Indirect inference. Journal of Applied Econometrics 8, S85–S118.

Kahn, R.J., Whited, T.M., 2018. Identification is not causality, and vice-versa. Review of Corporate Finance Studies 7, 1–21.

Lee, B.S., Ingram, B.F., 1991. Simulation estimation of time-series models. Journal of Econometrics 47, 197–205.

Michaelides, A., Ng, S., 2000. Estimating the rational expectations model of speculative storage: A Monte Carlo comparison of three simulation estimators. Journal of Econometrics 96, 231–266.

Newey, W.K., McFadden, D., 1994. Large sample estimation and hypothesis testing, in: Engle, R.F., McFadden, D.L. (Eds.), Handbook of Econometrics. Elsevier, Amsterdam. volume 4, pp. 2111–2245.

Pakes, A., Pollard, D., 1989. Simulation and the asymptotics of optimization estimators. Econometrica 57, 1027–1057.

Terry, S.J., Whited, T.M., Zakolyukina, A., 2023. Information versus investment. Review of Financial Studies 36, 1148–1191.

Copyright ©Toni M. Whited